API Connector - Developer Documentation

Sylvain Boutet

Version 1.0, 2025-01-03

Table of Contents

- Overview . . . 1
  - Main Use Cases . . . 1
- Module Architecture . . . 2
  - Main Models . . . 2
  - Hierarchy and Inheritance . . . 2
- Collection Configuration . . . 3
  - Main Fields . . . 3
  - Collection Variables . . . 3
  - Collection-Level Authentication . . . 3
- Connector Configuration . . . 4
  - Base Fields . . . 4
  - Variables and Parameters . . . 4
  - Request Body . . . 4
  - File Upload . . . 4
- Dynamic Variables . . . 6
  - Usage Syntax . . . 6
  - Priority Order . . . 6
  - Practical Examples . . . 6
- API Call Execution . . . 7
  - Execution Methods . . . 7
  - Available Variables in Crons . . . 7
  - Cron Execution Examples . . . 7
- Monitoring and Logs . . . 9
  - Statistics Fields . . . 9
  - Detailed Logs . . . 9
- File Upload . . . 10
  - Configuration . . . 10
  - Cron Usage Example . . . 10
  - Supported Formats . . . 10
- Security and Best Practices . . . 11
  - Credentials Management . . . 11
  - Error Handling . . . 11
  - Performance . . . 11
- No-Code Server Integrations . . . 12
  - Revolutionary Concept . . . 12
  - No-Code Server Workflow . . . 12
    - Step 1: API Configuration (100% graphical) . . . 12
    - Step 2: Scheduling (Odoo Interface) . . . 12

Step 3: Automatic Processing (Odoo Interface) . . . 12
No-Code Server Use Cases . . . 13
E-commerce Product Synchronization . . . 13
Customer Order Import . . . 13
Delivery Notification . . . 14
Automated Action Templates . . . 14
Template: Record Creation . . . 14
Template: Record Update . . . 14
No-Code Server Benefits . . . 15
Limitations and Recommendations . . . 15
Odoo Integration . . . 16
Automatic Triggers . . . 16
Compatible Odoo Models . . . 16
FAQ and Troubleshooting . . . 17
Common Errors . . . 17
Debug Logs . . . 17
Support and Community . . . 17
Configuration Export/Import . . . 18
Overview . . . 18
Configuration Export . . . 18
Export Usage . . . 18
Exported JSON File Structure . . . 18
Configuration Import . . . 19
Import Wizard . . . 19
Import Process . . . 19
Advanced Features . . . 20
Practical Use Cases . . . 20
Backup and Restore . . . 20
Environment Migration . . . 20
Configuration Sharing . . . 20
Versioning and History . . . 21
Cron Integration . . . 21
Programmatic Import . . . 21
Security and Best Practices . . . 21
Appendices . . . 23
Complete Example: Order Synchronization . . . 23
Appendix: Pre-configured Example Routes . . . 25
Collection: REST APIs Examples . . . 25
Get Posts . . . 25
Get Single Post . . . 25
Random Cat Fact . . . 25

🌐 Collection: International Public APIs . . . 25
Random User Generator . . . 25
↑ Collection: POST Request Examples . . . 26
Create New Post . . . 26
Create User Comment . . . 26
⇄ Collection: JSON-RPC Examples . . . 26
JSON-RPC Demo Call . . . 26
🖧 Collection: GraphQL Examples . . . 27
Rick and Morty GraphQL . . . 27
</> Collection: XML/SOAP Examples . . . 27
XML Demo Post . . . 27
🔒 Collection: Authentication Examples (Disabled) . . . 27
Basic Auth Demo . . . 27
Bearer Token Demo . . . 28
🛡 Security and Privacy . . . 28
▶ Using the Examples . . . 28
🔧 Customization . . . 28

Overview

The **API Connector** module is a Postman-like tool integrated into Odoo that allows you to configure and execute outbound API calls graphically and automatically.

Main Use Cases

- **Data Synchronization**: Send orders, invoices, contacts to external systems
- **E-commerce Integrations**: Sync inventory, prices, orders with platforms
- **Automatic Notifications**: Alert third-party systems on Odoo events
- **File Upload**: Send documents (PDF invoices, delivery notes) to APIs
- **Outbound Webhooks**: Trigger actions on external services

🏗 Module Architecture

Main Models

Model	Technical Name	Description
Collections	`api.collection`	Group of endpoints with shared configuration (authentication, base URL, variables)
Connectors	`api.connector`	Individual API endpoint configuration (URL, method, request body, authentication)
Call Logs	`api.call.log`	Detailed history of all API calls with response time, status, errors

Hierarchy and Inheritance

```
Collection (api.collection)
 ├─── Shared Configuration
 │    ├─── Base URL
 │    ├─── Authentication
 │    ├─── Default Headers
 │    └─── Collection Variables
 └─── Connectors (api.connector)
     ├─── Inherits from collection (optional)
     ├─── Specific configuration
     └─── Endpoint variables
```

⚙ Collection Configuration

Main Fields

Field	Type	Description
`name`	Char	Collection name (e.g., "Shopify API", "Stripe Webhooks")
`base_url`	Char	Shared base URL (e.g., "https://api.shopify.com/v1")
`auth_type`	Selection	Authentication type: None, Basic, Bearer, API Key, OAuth 2.0
`collection_variables`	Text (JSON)	Shared variables usable in all collection endpoints
`default_headers`	Text (JSON)	Default HTTP headers applied to all endpoints

Collection Variables

Collection variables allow centralizing reusable values:

```
{
    "api_version": "v2",
    "environment": "production",
    "store_id": "12345",
    "timeout": "30"
}
```

These variables can be used in URLs and request bodies with the `{variable_name}` syntax

Collection-Level Authentication

Type	Required Configuration
None	No authentication
Basic Auth	Username + Password
Bearer Token	Access token
API Key	Header name + API Key
OAuth 2.0	Client ID + Client Secret + Authorization and token URLs

🔌 Connector Configuration

Base Fields

Field	Type	Description
`name`	Char	Connector name (e.g., "Create Order", "Update Stock")
`collection_id`	Many2one	Parent collection (optional)
`api_type`	Selection	API type: REST, JSON-RPC, GraphQL, SOAP/XML
`method`	Selection	HTTP method: GET, POST, PUT, PATCH, DELETE
`base_url`	Char	Base URL (can inherit from collection)
`endpoint_path`	Char	Endpoint path (e.g., "/orders", "/products/{product_id}")

Variables and Parameters

Field	Type	Description
`url_parameters`	Text (JSON)	URL parameters for dynamic paths (e.g., `{"product_id": "123"}`)
`endpoint_variables`	Text (JSON)	Endpoint-specific variables (override collection variables)
`custom_headers`	Text (JSON)	Custom HTTP headers for this endpoint

Request Body

Type	Usage
JSON	Structured data, modern REST APIs
Form Data (URL-encoded)	Classic web forms
Form Data (Multipart)	File upload + data
XML	SOAP APIs, legacy services
Raw Text	Raw content, CSV, other formats
Binary (Base64)	Base64 encoded files

File Upload

Field	Type	Description
`attachment_ids`	Many2many	Odoo files to send (ir.attachment)
`file_field_name`	Char	File field name in the form (default: "ufile")

File upload requires **"Form Data (Multipart)"** body type

Dynamic Variables

Usage Syntax

Variables use the `{variable_name}` syntax and can be used in:

- URLs and endpoint paths
- Request body (JSON, XML, Raw)
- Custom headers

Priority Order

```
1. Dynamic variables (passed during execution)
2. Endpoint variables (endpoint_variables)
3. Collection variables (collection_variables)
```

Practical Examples

Collection variables

```
{
    "base_api": "v2",
    "store": "myshop",
    "format": "json"
}
```

Endpoint variables

```
{
    "resource": "orders",
    "action": "create"
}
```

Final URL

```
https://api.shopify.com/{base_api}/{store}/{resource}/{action}.{format}
→ https://api.shopify.com/v2/myshop/orders/create.json
```

▶ API Call Execution

Execution Methods

Method	Description
Graphical Interface	"Execute Call" button in the connector form
Cron Jobs	Scheduled execution via Odoo scheduled actions
Python Code	Direct call from Odoo business code

Available Variables in Crons

```
# Standard Odoo variables available
env          # Odoo environment
model        # Empty model
record       # Trigger record (may be empty)
records      # Multiple recordset (may be empty)
time         # Python time module
datetime     # Python datetime module
dateutil     # Python dateutil module
timezone     # Python timezone module
b64encode    # Base64 encoding function
b64decode    # Base64 decoding function
_logger      # Logger for server logs
log          # Log function in ir.logging
UserError    # Exception for user messages
Command      # Namespace for x2many commands
```

Cron Execution Examples

Simple execution

```
connector = env['api.connector'].browse(123)
result = connector.execute_call()
```

With dynamic variables

```
connector = env['api.connector'].browse(123)
result = connector.execute_call({
    'order_id': record.id,
    'customer_name': record.partner_id.name,
    'date': datetime.date.today().strftime('%Y-%m-%d')
})
```

Batch processing

```
collection = env['api.collection'].browse(456)
orders = env['sale.order'].search([('state', '=', 'sale')])

for order in orders:
    for connector in collection.connector_ids:
        result = connector.execute_call({
            'order_id': order.id,
            'total': order.amount_total
        })
        if not result.get('success'):
            _logger.error("Sync failed for order %s", order.name)
```

📈 Monitoring and Logs

Statistics Fields

Field	Type	Description
`total_calls`	Integer	Total number of calls made
`success_calls`	Integer	Number of successful calls (2xx status)
`success_rate`	Float	Success rate (0.0 to 1.0)
`last_call_date`	Datetime	Last call date
`last_response_status`	Integer	HTTP status code of last response
`last_response_time`	Float	Response time in seconds
`last_response_data`	Text	Last response data

Detailed Logs

Each API call generates an `api.call.log` record containing:

- Precise date and time
- User who triggered the call
- Request parameters
- Request and response size
- Execution time
- Status code and error message if any

File Upload

Configuration

For file upload, configure:

1. **Body type**: "Form Data (Multipart)"
2. **Field name**: "ufile" (or according to target API)
3. **Files**: Select Odoo attachments

Cron Usage Example

```
# Create attachment from file
attachment = env['ir.attachment'].create({
    'name': 'invoice.pdf',
    'datas': b64encode(pdf_content),
    'mimetype': 'application/pdf'
})

# Add to connector and execute
connector = env['api.connector'].browse(789)
connector.write({'attachment_ids': [(4, attachment.id)]})
result = connector.execute_call({
    'invoice_id': invoice.id,
    'customer_ref': invoice.partner_id.ref
})
```

Supported Formats

The module automatically handles:

- **Single file**: `ufile` field
- **Multiple files**: `ufile`, `ufile[1]`, `ufile[2]`, etc. fields
- **MIME types**: Automatic detection from attachment
- **File names**: Preservation of original names

Security and Best Practices

Credentials Management

- Store tokens and API keys securely
- Use environment variables for sensitive values
- Limit access rights to critical collections
- Enable SSL verification by default

Error Handling

Recommended pattern

```
try:
    connector = env['api.connector'].browse(connector_id)
    result = connector.execute_call(variables)

    if result.get('success'):
        _logger.info("API sync successful")
        # Process response
    else:
        _logger.error("API error: %s", result.get('error'))
        # Handle error

except Exception as e:
    _logger.error("Sync failed: %s", str(e))
    raise UserError("Synchronization failed: %s" % str(e))
```

Performance

- Configure appropriate timeouts (30s default)
- Use collections to reduce duplication
- Monitor logs to identify slow calls
- Implement retry patterns for unstable APIs

🪄 No-Code Server Integrations

Revolutionary Concept

The API Connector module enables creating **complete integrations without writing Python files**. All configuration is done via the standard Odoo interface by combining:

- **API Connectors**: Graphical API call configuration
- **Odoo Crons**: Scheduling via standard interface
- **Automated Actions**: Response processing via interface

No-Code Server Workflow

3-step process

```
1. □ API Connector → Programmed external API call
2. □ Odoo Cron → Automatic periodic execution
3. □ Automated Action → Response processing
```

Step 1: API Configuration (100% graphical)

💡

- **URL and authentication**: API Connector interface
- **Dynamic variables**: Automatic substitution
- **Request body**: JSON with `{variable_name}` variables
- **Monitoring**: Integrated automatic logs

Step 2: Scheduling (Odoo Interface)

Minimal code in cron (via interface)

```
# Configuration in Settings > Technical > Automation > Scheduled Actions
connector = env['api.connector'].browse(123)  # ID from interface
result = connector.execute_call({
    'date': datetime.date.today().strftime('%Y-%m-%d'),
    'company_id': env.company.id
})
```

Step 3: Automatic Processing (Odoo Interface)

Automated action on api.connector (via interface)

```
# Configuration in Settings > Technical > Automation > Automated Actions
# Trigger: "On Update of a Record"
# Condition: last_response_success == True

import json

if record.last_response_data:
    response = json.loads(record.last_response_data)

    # Process response according to integration type
    if record.name == 'Sync Products':
        for product_data in response.get('products', []):
            env['product.product'].create({
                'name': product_data['name'],
                'default_code': product_data['sku'],
                'list_price': product_data['price']
            })

    elif record.name == 'Import Orders':
        for order_data in response.get('orders', []):
            partner = env['res.partner'].search([('email', '=', order_data
['customer_email'])], limit=1)
            if partner:
                env['sale.order'].create({
                    'partner_id': partner.id,
                    'amount_total': order_data['total']
                })
```

No-Code Server Use Cases

E-commerce Product Synchronization

Component	Configuration	Result
API Connector	URL: `https://api.shopify.com/products`	Retrieves product catalog
Daily Cron	Execution at 2 AM	Automatic API call
Automated Action	Trigger on `last_response_data`	Create/update Odoo products

Customer Order Import

Component	Configuration	Result
API Connector	URL: `https://api.woocommerce.com/orders`	Retrieves new orders

Component	Configuration	Result
Hourly Cron	Execution every hour	Order checking
Automated Action	Condition: new orders	Create Odoo `sale.order`

Delivery Notification

Component	Configuration	Result
API Connector	URL: `https://api.carrier.com/notify`	Carrier notification
Automated Action	Trigger: `stock.picking` validated	Automatic API call
Response	Confirmation received	Update delivery status

Automated Action Templates

Template: Record Creation

Generic automated action

```
# Condition: last_response_success == True and 'create_' in record.name
import json

if record.last_response_data:
    response = json.loads(record.last_response_data)

    # Generic mapping from endpoint_variables
    mapping_config = json.loads(record.endpoint_variables or '{}')
    target_model = mapping_config.get('target_model', 'res.partner')
    field_mapping = mapping_config.get('field_mapping', {})

    for item in response.get('data', []):
        values = {}
        for odoo_field, api_field in field_mapping.items():
            if api_field in item:
                values[odoo_field] = item[api_field]

        if values:
            env[target_model].create(values)
            _logger.info(f"Created {target_model} from API response")
```

Template: Record Update

Update automated action

```
# Condition: last_response_success == True and 'update_' in record.name
import json

if record.last_response_data:
    response = json.loads(record.last_response_data)
    mapping_config = json.loads(record.endpoint_variables or '{}')

    target_model = mapping_config.get('target_model')
    search_field = mapping_config.get('search_field', 'id')

    for item in response.get('data', []):
        search_value = item.get(search_field)
        if search_value:
            existing_record = env[target_model].search([(search_field, '=',
search_value)], limit=1)
            if existing_record:
                update_values = {}
                for odoo_field, api_field in mapping_config.get('field_mapping',
{}).items():
                    if api_field in item:
                        update_values[odoo_field] = item[api_field]

                if update_values:
                    existing_record.write(update_values)
```

No-Code Server Benefits

For business users: * Entirely graphical configuration * No custom development required * Reuse of existing templates * Maintenance via Odoo interface

For developers: * Creation of reusable templates * Less custom code to maintain * Integration standardization * Focus on complex business logic

Limitations and Recommendations

This approach is suitable for: * Simple integrations (basic CRUD) * Predictable structured data * Light transformations

Use custom code for: * Complex business logic * Advanced data transformations * Sophisticated error handling * Critical performance

Odoo Integration

Automatic Triggers

The module can be integrated with:

- **Cron jobs**: Scheduled execution
- **Automated actions**: Response processing (no-code server)
- **Odoo workflows**: Automatic actions on models
- **Incoming webhooks**: Via complementary `json_rpc_api` module
- **Server actions**: Buttons in views

Compatible Odoo Models

The module can synchronize data from/to all Odoo models:

- `sale.order`: Orders
- `account.move`: Invoices
- `stock.picking`: Deliveries
- `res.partner`: Contacts
- `product.product`: Products
- And all custom models

⍰ FAQ and Troubleshooting

Common Errors

Error	Cause	Solution
Timeout	Slow or unavailable API	Increase timeout, check API
SSL Error	Invalid certificate	Temporarily disable verify_ssl
401 Unauthorized	Incorrect authentication	Check tokens and credentials
Variables not substituted	Incorrect syntax	Use `{variable}` and check spelling

Debug Logs

To enable detailed logs:

```
import logging
_logger = logging.getLogger(__name__)
_logger.setLevel(logging.DEBUG)
```

Support and Community

- **Documentation**: Check Documentation tabs in the interface
- **Logs**: "Call Logs" menu for detailed history
- **Testing**: Use "Execute Call" button to test
- **Variables**: Check substitution in logs

📥 Configuration Export/Import

Overview

The API Connector module allows exporting and importing configurations in JSON format to facilitate backup, sharing, and migration of configurations between environments.

Configuration Export

Type	Method	Exported Content
Connector	"Export Config" button in the form	Complete connector configuration + collection reference
Collection	"Export Config" button in the form	Collection configuration + all associated connectors

Export Usage

From the interface

```
1. Open a connector or collection
2. Click "Export Config" in the button box
3. JSON file downloads automatically
```

Exported JSON File Structure

Connector export example

```
{
  "name": "Create Order API",
  "api_type": "rest",
  "method": "POST",
  "endpoint_path": "/orders",
  "auth_type": "bearer",
  "auth_token": "sk_test_xxxxx",
  "request_body_type": "json",
  "request_body": "{\"order\": {\"total\": \"{total}\"}}",
  "collection_name": "Shopify API",
  "_metadata": {
    "export_date": "2025-01-03T10:30:00",
    "export_user": "Admin",
    "odoo_version": "18.0",
    "module_version": "1.0.0",
    "type": "api_connector"
  }
}
```

Collection export example

```
{
  "name": "Shopify API",
  "base_url": "https://myshop.myshopify.com/admin/api/2023-01",
  "auth_type": "bearer",
  "auth_token": "shpat_xxxxx",
  "collection_variables": "{\"format\": \"json\"}",
  "connectors": [
    {
      "name": "Get Products",
      "method": "GET",
      "endpoint_path": "/products.{format}",
      "use_collection_url": true
    },
    {
      "name": "Create Order",
      "method": "POST",
      "endpoint_path": "/orders.{format}",
      "request_body_type": "json"
    }
  ],
  "_metadata": {
    "export_date": "2025-01-03T10:30:00",
    "export_user": "Admin",
    "odoo_version": "18.0",
    "module_version": "1.0.0",
    "type": "api_collection",
    "connectors_count": 2
  }
}
```

Configuration Import

Import Wizard

Import uses a graphical wizard accessible via: - **Menu**: Configuration → Import Configuration - **Button** in connector/collection lists

Import Process

Import steps

```
1. Select the exported JSON file
2. Automatic content preview
3. Choose configuration type (auto-detected)
4. For connectors: choose target collection (optional)
5. Click "Import Configuration"
6. Redirect to created record
```

Advanced Features

- **Auto-detection**: Configuration type is automatically detected
- **Validation**: Data integrity verification before import
- **Smart search**: Collections are found by name
- **Automatic inheritance**: Imported connectors inherit from their collection
- **Error handling**: Explicit error messages when problems occur

Practical Use Cases

Backup and Restore

Backup workflow

```
# 1. Manual export from interface
# 2. JSON files storage
# 3. Import when restore needed
```

Environment Migration

Development → Production Environment

```
1. Export collections from DEV
2. Transfer JSON files
3. Import into PROD environment
4. Adjust URLs and tokens if necessary
```

Configuration Sharing

Ready-to-use configuration distribution

```
1. Create collection with all API endpoints
2. Export complete configuration
3. Share JSON file with team
4. One-click import in each environment
```

Versioning and History

Version management

```
1. Export before important modifications
2. Version naming (e.g., shopify_v1.2.json)
3. Storage in version control system
4. Rollback possible if problems occur
```

Cron Integration

Programmatic Import

Import via Python code

```python
# Import a connector
import json

config_data = {
    "name": "Auto Generated Connector",
    "api_type": "rest",
    "method": "POST",
    "endpoint_path": "/webhook",
    "_metadata": {
        "type": "api_connector",
        "export_date": "2025-01-03T10:30:00",
        "export_user": "System",
        "odoo_version": "18.0",
        "module_version": "1.0.0"
    }
}

connector_id = env['api.connector'].import_config(config_data, collection_id=123)
```

Import complete collection

```python
# Reading from file
with open('/path/to/collection_export.json', 'r') as f:
    config_data = json.load(f)

result = env['api.collection'].import_config(config_data)
```

Security and Best Practices

Sensitive data in exports: * Tokens and passwords are included in exports * Store JSON files securely * Use environment variables for credentials * Review exports before external sharing

Recommendations: * **Naming**: Use explicit names (api_shopify_prod_2025.json) * **Documentation**: Add comments in descriptions * **Testing**: Test imports on test environment * **Backup**: Regularly export your critical configurations

🚀 Appendices

Complete Example: Order Synchronization

Collection configuration

```
# Collection "Shopify API"
base_url: "https://myshop.myshopify.com/admin/api/2023-01"
auth_type: "Bearer Token"
auth_token: "shpat_xxxxx"
collection_variables: {
    "format": "json",
    "fields": "id,name,total_price,customer"
}
```

Connector configuration

```
# Connector "Create Order"
endpoint_path: "/orders.{format}"
method: "POST"
request_body_type: "json"
endpoint_variables: {
    "source": "odoo_erp"
}
```

JSON request body

```
{
    "order": {
        "line_items": [
            {
                "variant_id": "{product_variant_id}",
                "quantity": "{quantity}",
                "price": "{unit_price}"
            }
        ],
        "customer": {
            "email": "{customer_email}",
            "first_name": "{customer_firstname}",
            "last_name": "{customer_lastname}"
        },
        "source_name": "{source}",
        "note": "Order created from Odoo on {creation_date}"
    }
}
```

Cron execution

```
# Daily synchronization cron
for order in env['sale.order'].search([('state', '=', 'sale'), ('synced', '=',
False)]):
    connector = env['api.connector'].browse(123)  # Connector ID

    result = connector.execute_call({
        'product_variant_id': order.order_line[0].product_id.shopify_variant_id,
        'quantity': order.order_line[0].product_uom_qty,
        'unit_price': order.order_line[0].price_unit,
        'customer_email': order.partner_id.email,
        'customer_firstname': order.partner_id.firstname,
        'customer_lastname': order.partner_id.lastname,
        'creation_date': order.date_order.strftime('%Y-%m-%d')
    })

    if result.get('success'):
        order.write({'synced': True, 'shopify_order_id': result.get('data', {}).get
('id')})
        _logger.info("Order %s synchronized to Shopify", order.name)
    else:
        _logger.error("Failed to sync order %s: %s", order.name, result.get('error'))
```

Appendix: Pre-configured Example Routes

The API Connector module includes a set of pre-configured example routes to facilitate onboarding and demonstrate the different types of APIs supported.

Collection: REST APIs Examples

Base URL: `https://jsonplaceholder.typicode.com`

Get Posts

Type	REST GET
Endpoint	`/posts`
Description	Retrieves the complete list of demo articles
Response	JSON Array with id, title, body, userId

Get Single Post

Type	REST GET
Endpoint	`/posts/{id}`
Parameters	`{"id": "1"}`
Description	Retrieves a specific article by its ID
Response	JSON Object with article details

Random Cat Fact

Type	REST GET
URL	`https://catfact.ninja/fact`
Description	Generates a random fun fact about cats
Response	`{"fact": "…", "length": 123}`
Usage	Simple and reliable external API demonstration

Collection: International Public APIs

Random User Generator

Type	REST GET
URL	`https://randomuser.me/api/`
Description	Generates a complete fictional user
Response	Complete data (name, photo, address, contact)

Usage	Perfect for testing and demonstrations

↑ Collection: POST Request Examples

Base URL: `https://jsonplaceholder.typicode.com`

Create New Post

Type	REST POST
Endpoint	`/posts`
Body Type	JSON
Body	`{"title": "Demo Post", "body": "This is a demo post content", "userId": 1}`
Description	Creates a new article via JSON POST request
Response	Created object with generated ID

Create User Comment

Type	REST POST
Endpoint	`/comments`
Body Type	JSON
Body	`{"name": "Demo User", "email": "demo@example.com", "body": "This is a demo comment", "postId": 1}`
Description	Creates a comment associated with an article
Response	Created comment with generated ID

⇄ Collection: JSON-RPC Examples

JSON-RPC Demo Call

Type	JSON-RPC
URL	`https://jsonplaceholder.typicode.com/posts`
Body Type	JSON
Body	`{"jsonrpc": "2.0", "method": "get_demo_data", "params": {"format": "json", "count": 5}, "id": 1}`
Description	Demonstrates JSON-RPC protocol with method and parameters
Usage	Illustration of standard JSON-RPC format

Collection: GraphQL Examples

Rick and Morty GraphQL

Type	GraphQL
URL	https://rickandmortyapi.com/graphql
Body Type	JSON
Query	`{ character(id: 1) { name status species } }`
Description	GraphQL query on Rick and Morty API
Response	`{"data": {"character": {"name": "Rick Sanchez", "status": "Alive", "species": "Human"}}}`
Usage	Demonstration of structured GraphQL queries

Collection: XML/SOAP Examples

XML Demo Post

Type	SOAP/XML
URL	https://jsonplaceholder.typicode.com/posts
Body Type	XML
Body	XML product structure with name, price, category
Description	Sends XML data via POST request
Headers	`Content-Type: application/xml`
Usage	Illustration of XML/SOAP exchanges

Collection: Authentication Examples (Disabled)

These routes are disabled by default as they require valid tokens.

Basic Auth Demo

Type	REST GET with Basic Auth
URL	https://jsonplaceholder.typicode.com/posts/1
Auth	Basic (demo_user / demo_pass)
Description	Basic authentication demonstration
Status	Disabled (active=False)

Bearer Token Demo

Type	REST GET with Bearer Token
URL	`https://jsonplaceholder.typicode.com/posts/1`
Auth	Bearer (demo_bearer_token_123)
Description	Bearer authentication demonstration
Status	Disabled (active=False)

Security and Privacy

All example APIs are secured for public demonstrations:

- **No personal data exposed**: APIs do not return caller information
- **No IP address revealed**: Unlike httpbin.org, these APIs don't log connection details
- **Legitimate public APIs**: Official test services and recognized community APIs
- **Fictional data only**: All examples use demonstration data

Using the Examples

To use these examples:

1. **Installation**: Routes are created automatically during module installation
2. **Navigation**: Menu API Connector → Collections → Select a collection
3. **Testing**: Click on a connector and use the "Execute Call" button
4. **Customization**: Duplicate and modify according to your needs
5. **Production**: Replace test URLs with your real APIs

Customization

To adapt these examples to your needs:

```
# Modify an existing route
connector = self.env['api.connector'].search([('name', '=', 'Random Cat Fact')])
connector.write({
    'base_url': 'https://your-api.com/endpoint',
    'custom_headers': '{"Authorization": "Bearer YOUR_TOKEN"}',
    'active': True
})

# Create a new route based on an example
new_connector = connector.copy({
    'name': 'My Custom API',
    'base_url': 'https://my-api.com/data',
    'collection_id': collection.id
})
```

Version: 1.0 - API Connector Module for Odoo 18 **Author**: Sylvain Boutet - IROKOO **Date**: 2025-01-03